

NetAssist

*Proof of Concept by Syed Faisal Ali
University of Texas at Dallas*

May 2025

Resources

- Github Repository: <https://github.com/SyedFaisalAli/NetAssist>

Introduction

Project Overview

NetAssist is a network agent that can provide assistance and interact with the system it's running on, locally, without the need for internet connection. The goal is to utilize small language models (SLM) such as Qwen 3, and enhance the chat flow with relevant context through a network model context protocol (MCP) server. This allows the SLM to request information about the current network through function calls, which can be used to steer the SLM to provide the user relevant responses and actions. For the proof of concept, the assistant only works to troubleshoot on Linux devices that utilizes systemd-networkd, along with standard tools such as `ip` and `ping`.

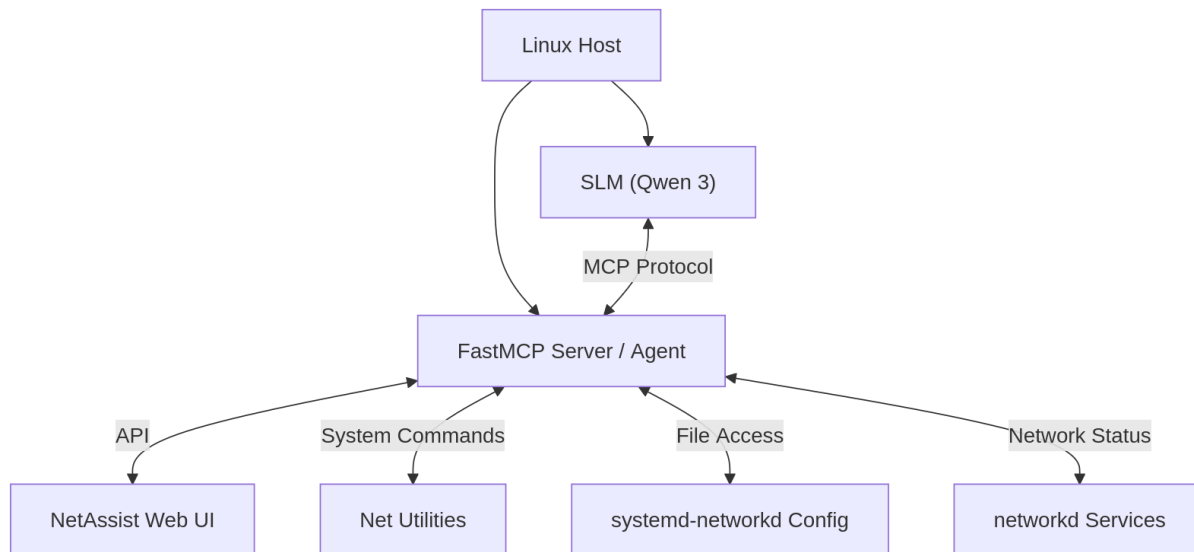
Background

Running SLM on the device for targeted tasks can help lower the barrier when it comes to network management. During installation of a device out in the field or on a personal computer, there are multiple points of failure such as firewall, routing, poor wire/wireless connectivity, and ISP degradations.

Critically, when there's an issue with connecting to the internet, the user can't rely on a large language model (LLM) for assistance. The LLM would also not have the context to the network configuration in terms of the user's issues or requests unless it can interface with an MCP server on the device. Especially for Linux devices, there are multiple networking services and options available, making it difficult to request help unless the user knows their system.

For customer support and installers, having an SLM augmented by appropriate tooling and context can ease troubleshooting installation by allowing the agent to perform sequential tasks faster as it will be given feedback by each tool it requests. In addition, the user can request information using natural language, and the SLM can interact, prompting for more info as needed.

Proof of Concept



Architecture

- Linux host: NVIDIA Orin Nano Jetson development board.
 - SLM: Qwen 3 1.7B 4-bit Quantization
 - Served locally through Ollama under Docker, providing an OpenAI-compatible API endpoint.
 - MCP: Using the library, FastMCP, a custom API server in Python, was written to expose the underlying host network tools and provide interpretation according to the MCP standard. The function tools that were exposed by this server involve:
 - Net Utilities, such as `ping` and `ip`, made through python `subprocess` calls.
 - File access to networkd config files under `/usr/lib/systemd/network/*`
 - Networkd service status, primarily revolving around making subprocess calls to `networkctl`.
 - Programmatic retrievals of network status through `netifaces` library and `psutils`.

- NetAssist Web UI is a React-based frontend that was written to interact with the FastMCP server to retrieve similar responses that would be sent to the MCP. The chat messages between Ollama and the MCP server are managed through the bridge agent, using websockets to pass updates and messages to the Web UI.

Challenges

In finding a suitable SLM model, smaller parameters and quantization means lower accuracy. Especially when it comes to networking knowledge, more details about a configuration may have to be provided using RAG to compensate. Ultimately, Qwen 3 came out recently, relative to the time of this report, which adds reasoning capabilities and fair baseline networking training.

At the cost of time “thinking”, it can help identify solutions and missing details from the user with minimal prompting, so it was chosen as the default model.

Contributions

This is a solo project, with the frontend written using React, and the MCP and agent written in Python using FastAPI/FastMCP. The following describes the frameworks and libraries used:

- Agent and MCP server:
 - FastMCP, which registers the functions tools to be exposed by the underlying FastAPI backend.
 - The FastAPI backend is extended to serve the generated static files of the React frontend as well as simple non-MCP endpoints, such as to retrieve the list of interfaces to show on the frontend.
- Frontend (Chat UI)
 - Used React to control the DOM elements and Mantine to provide existing reusable components and styling.

Conclusion

SLM can be useful as a base model once it's augmented to select tooling, but it relies on well-formatted outputs from said tools to help steer it and provide summarization. With good prompting, associated tagging for functions exposed from the MCP, and the right model with relevant training data, it can help with network provisioning and configuration in a product out in the field or on a personal computer.

While there are still risks with hallucinations, appropriate guardrails in the function calls implemented in the MCP server can help ensure accurate responses and deterministic behavior. As SLM memory usage lowers, one can expect these models to be embedded into devices to handle first line of requests for a task rather than rely directly on LLM providers.